

ZHAW – Zurich University of Applied Sciences

School of Engineering

Drone Control Tuning

Development of an Offline Betaflight Drone Control Tuner

Autor:	Yuri Bianchi, Dario Jurietti and Janick Dort
Modul / PA:	Project Thesis
Supervisor:	Michael Ernst Peter
Co-Supervisor:	Dr. Prof. Ruprecht Altenburger

Winterthur, 3. Dezember 2025

1 Abstract

This project focuses on the development of a tool for offline tuning of gyro controllers for FPV racing drones based on the open-source firmware Betaflight. The quality of the gyro control loop has a direct influence on flight stability, the suppression of disturbances such as prop wash, and the agility of the drone. Until now, rate controller tuning has mostly been performed through time-consuming trial and error.

The aim of the project is to develop a modular, open-source solution for offline tuning based on an existing proof of concept. The code will be implemented in MATLAB, accompanied by a lightweight Python version, and supplemented with detailed Markdown documentation and examples on GitHub. The project aims to provide the community with a simple way to efficiently optimise drone parameters and thereby improve flight performance.

The results will be published in a public GitHub repository and documented in a report. In addition to the software development, the project also includes practical work with FPV drones to validate the developed methods.

2 Foreword

This document summarizes the results of our project thesis, which we worked on during the fall semester 2025 at the ZHAW School of Engineering. For us as systems engineering students, this project provided an ideal opportunity to apply advanced control theory and signal processing methods to a real, highly dynamic system. FPV drones present an interesting engineering challenge due to their highly dynamic behaviour and the need for precise control. This motivated us to investigate mathematical modelling, control engineering aspects, and offline tuning methods aimed at improving flight performance.

Our motivation was to translate the theoretical control concepts we learned over the past three years to a real, highly dynamic system. We also aimed to develop a tool that not only meets engineering standards but also provides real value to the FPV community by replacing subjective tuning with objective data analysis.

This work was carried out at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. We would like to express our gratitude to our supervisor, Michael Peter, for his guidance and support throughout this project. His expertise, constructive feedback, and willingness to take the time to assist us in developing complex algorithms were invaluable.

Inhaltsverzeichnis

1	Abstract	2
2	Foreword	2
3	Introduction	5
3.1	Initial situation	5
3.2	Task and Objective	5
4	Preparation	6
4.1	Chirp	6
4.1.1	Chirp Concept	6
4.1.2	Chirp on a drone	6
4.1.3	Chirp programming	7
5	Theory	8
5.1	Signal Processing	8
5.1.1	Data Import	8
5.1.2	Handle High-Resolution Data	8
5.1.3	Time Conversion and Sampling Rate Verification	8
5.1.4	Application of Rotational Filtering (Apply Rotfiltfilt)	8
5.1.5	Estimation of Frequency Response	8
5.2	Control System	9
5.2.1	Transfer Function Estimation	9
5.2.2	Closed Loop Calculation	9
5.2.3	Filters and Controllers	9
5.2.4	Compensation of the I-Term	9
5.3	Evaluation	10
5.3.1	Step Response	10
5.3.2	Spectra Analysis	10
5.3.3	Spectrum Analysis	10
6	Challenges and Methodological Approach	11
6.1	Simulations	11
6.1.1	Simulation of Betaflight Controller Order	11
6.1.2	Simulation PID Controller	11
6.1.3	Simulation Frequency Response	11
6.1.4	Simulation Spectra	12
6.1.5	Simulation Spectrogram	12
6.2	Building an FPV drone	13
6.3	Tuning an FPV drone	13
6.4	Decode	15
6.5	Code Structure and programming style	16
6.5.1	OOP vs. Functional Approach	16
6.5.2	Final Structure Concept	18
6.6	Conversion MATLAB to Python	20
6.7	Python in MATLAB	20
6.8	Transfer MATLAB Library to Python	20
7	Results	21

8	Discussion and next steps	21
8.1	Discussion	21
8.2	Next Steps	21
8.3	Conclusion	21
9	Directories	22

3 Introduction

Text...

3.1 Initial situation

Text...

3.2 Task and Objective

Text...

4 Preparation

4.1 Chirp

4.1.1 Chirp Concept

A chirp is an input signal that sweeps continuously over a defined range (e.g. from a low frequency to a high frequency). This signal excites the system on multiple frequencies in one run and thus lets you observe the frequency response and step response, the foundation of controller tuning. Mathematically, a chirp signal can be represented as a sinusoidal wave with instantaneous frequency that varies linearly or exponential with time. For more information about the mathematical background, please have a look in the document [Chirp Signal](#).

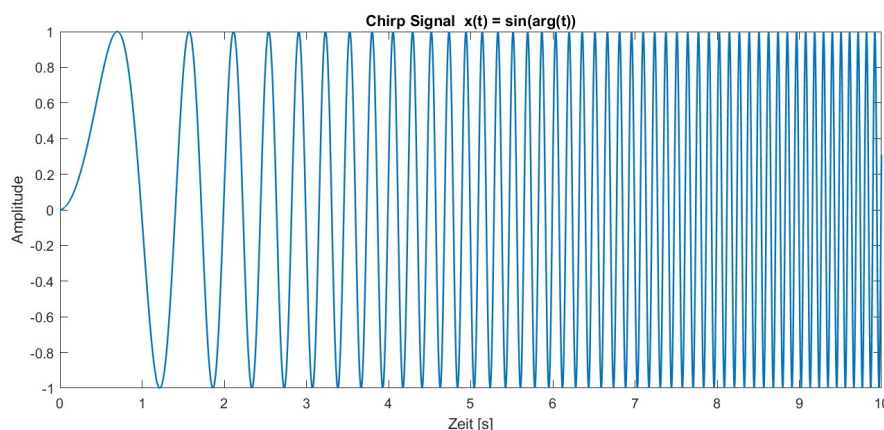


Abbildung 1: Chirp Signal example

4.1.2 Chirp on a drone

In the context of tuning a drone's flight controller, an exponential sweep is used. This is due to the dominant low-frequency dynamics, where low frequencies exhibit more predictable responses, allowing the sweep to allocate more time to higher frequencies for better resolution of system behavior. Additionally, multirotor vibration resonances are typically spaced logarithmically, making exponential sweeps more suited for efficient excitation across the frequency range.

The process involves these steps:

1. **Signal Injection:** The chirp signal is added to the pilot's stick input. This composite signal becomes the setpoint for the PID controller.
2. **System Response:** The drone attempts to follow this rapidly changing setpoint. The motors react, and the drone will oscillate at varying frequencies corresponding to the chirp signal.
3. **Data Logging:** While the chirp is active, high-resolution data from the gyroscope and the motor outputs are recorded using Betaflight's Blackbox feature. This data captures how the drone responds to the control inputs at different frequencies.

The recorded log file, containing the input chirp and the corresponding gyroscope output, is then used for offline analysis to determine the frequency response of the system.

4.1.3 Chirp programming

The implementation of the chirp signal in Betaflight is straightforward. You call the function `chirpInit()` and pass:

- a struct of type `chirp_t` defined in `chirp.h`
- starting frequency
- end frequency
- signal length
- loop-time in microseconds

This initialises the values in the data struct and:

- converts loop period from microseconds to seconds
- calculates the number of samples in the signal duration
- calculates the exponential growth factor
- precomputes the constants used to get the phase from the frequency evolution
- calls `chirpReset()` function

The `chirpReset()` function sets:

- internal counter to 0
- bool `isFinished` to false

then calls the function `chirpResetSignals()` to set signal-related fields to 0

Once the chirp is initialised use the `chirpUpdate()` function once per loop iteration. This function checks:

- if bool `isFinished` is true -> return false
- else if internal counter is equal to the number of samples inside the chirps period
-> set bool `isFinished` true, call `chirpResetSignals()`, return false

if none of the above is true the ...

5 Theory

5.1 Signal Processing

Text....

5.1.1 Data Import

Text....

5.1.2 Handle High-Resolution Data

Text...

5.1.3 Time Conversion and Sampling Rate Verification

Text...

5.1.4 Application of Rotational Filtering (Apply Rotfiltfilt)

Text...

5.1.5 Estimation of Frequency Response

Text...

5.2 Control System

Text...

5.2.1 Transfer Function Estimation

Text...

5.2.2 Closed Loop Calculation

Text...

5.2.3 Filters and Controllers

Text...

5.2.4 Compensation of the I-Term

Text...

5.3 Evaluation

Text...

5.3.1 Step Response

Text...

5.3.2 Spectra Analysis

Text...

5.3.3 Spectrum Analysis

Text...

6 Challenges and Methodological Approach

6.1 Simulations

At the beginning of our thesis, we started to create simulations. We used these simulations to deepen our understanding of the existing code and to understand the used methods. The simulations were crucial, but also very time intensive. They formed the basis for all later analysis steps and helped us understanding the existing functions before we used them further.

6.1.1 Simulation of Betaflight Controller Order

For analysis and for better understanding of the Betaflight control system, we recreated the complete control structure in MATLAB. You will find the corresponding script in the attachments. In the simulation, we attempted to reproduce the step response of the recorded data by using the same PID parameters and filter settings as in the real system. The different filter were created according to the documentation [Filter and Controller](#). We implemented the filters both continuously and discretely, so that we could compare the two types. The PI- and D-Controller were built in the same way. For the transfer function of the plant we used a simplified version out of PT '2 elements and dead time, based on the approaches from the documentation [Calculation Closed Loop](#). In addition to the MATLAB code, we created a Simulink model, in which we tested the step response of our simulation of the Betaflight control system. The simulated answer got close to the step response of the real drone. For us, this was the confirmation, that we got the controller order correct.

6.1.2 Simulation PID Controller

To deepen our understanding of the different controller components, we created an additional MATLAB script. It allowed us to combine different controller types with simple plant models. In the program you can switch between P, I, PI, PD and PID controllers, and between basic plant types. Through this, we were able to observe how each controller component influences the overall system behaviour. The script calculates the open and closed loop of the chosen settings and generates Bode plots, Nyquist diagrams and step responses. This helped us to understand and visualize the effect of each component. Through that, our overall understanding of controller behaviour significantly improved.

6.1.3 Simulation Frequency Response

To analyse the frequency response of a system based on input and output data, we created an additional simulation. In this simulation we created a chirp signal on which we added noise for a more realistic measurement condition. Using these signals, we implemented different methods to estimate the system's transfer function. First, we calculated a basic frequency response using the plain **F**ast **F**ourier **T**ransformation (FFT). That got us a good benchmark. After that, we applied different signal processing methods. First we used the Welch method to improve the estimation by averaging over overlapping segments. The implementation of the Welch **F**requency **R**esponse **F**unction (FRF) follows the theory described in the documentation [Estimate Frequency Response](#). This allowed us to compare the plain FFT approach with the more robust Welch estimator. In the next step we implemented a rotated-signal approach, where the chirp is demodulated to baseband before filtering. For this part we used the zero-phase filtering method `apply_rotfiltfilt`, as described in the documentation [Apply Rotfiltfilt](#). After that we compared the original bode plot to the signal processed one '2, to see how the rotation technique improves the noise robustness.

Overall, this script helped us understand how noise affects the estimated transfer function and how we can increase the robustness of the signals.

6.1.4 Simulation Spectra

To better understand the different components use to create a meaningful spectrum, we developed a simulation based on the system used in the given code. In this script we generated a noisy sinusoidal input signal out of which we created different spectra. In one case we applied a window function and in the other we did not use any window. By splitting the signal into several overlapping segments, we were able to observe how averaging across segments reduces noise and stabilises the resulting spectrum. The implementation is described in detail in the document [Spectra Analysis](#). Overall, this exercise helped us understand how windowing, overlap and segment length affect the spectral estimation. Also how these methods can increase robustness and reduce spectral leakage.

6.1.5 Simulation Spectrogram

For better understanding of the analysis how the frequency content of the excitation signal changes with both time and thrust, we developed an further MATLAB simulation. In this script, we generated a noisy sinusoidal input signal and a slowly varying thrust signal. The input signal is then split up in different segments and windowed with a Hann window. After this the signal is transformad using the FFT in order to obtain a time-frequency representation. At first, we arranged the segment spectra along the time axis and in e second step, along the thrust levels. So we got two spectrograms, one is showing the amplitude over frequency and time, and the other one is showing us the amplitude over frequency and thrust. Through this visualisation, we can identify the dominant frequency components depending on thrust. The underlying method is described in detail in the document [Spectrogram Analysis](#).

6.2 Building an FPV drone

Besides the understanding of the code and further developing the proof of concept, building a drone and set it up correctly bridges the gap between the digital domain, where algorithms and simulations are developed and the physical reality of drone operation. By getting in touch with the hardware, it is easier for us to understand what practical challenges exist for pilots and the community.



Stock Photo

Abbildung 2: IFLIGHT aosmini build

Therefore a three inch FPV drone was built from the ground up. This work included:

- creating schemes to see which connections were required
- soldering all the components as the motors, receiver, video system, logger and gps module to the flight controller
- attaching everything to the frame, so that nothing generates unnecessary oscillations during flight
- double check all soldering connections and screws
- setting everything up in betafight
- testfly the drone

The biggest challenge was to achieve good soldering connections ...

6.3 Tuning an FPV drone

As part of understanding the workflow we tuned an FPV drone by ourselves. The goal was to adjust the PID parameters so that we achieve a fast rising step response without overshoot, a lower compliance than before and a rounded shaped Sensitivity. Also to mention is, that the axes roll and pitch should be tuned as similar to each other as possible. This is because you want the drone to maintain a consistent

and predictable response, so roll and pitch behave similarly for smoother control and a balanced flight experience.

Initially, we deliberately applied a poor tuning configuration to observe its effects. During the first field test with this setup, it quickly became apparent that the tuning was excessively poor. The drone began oscillating aggressively and climbed rapidly. As the drone was out of control, the kill switch had to be activated, resulting in a crash that broke one of the four arms. Fortunately, the damage was minor and repaired quickly.

After this incident, we decided not to repeat flights with intentionally bad tuning. Instead, we continued using the default Betaflight parameters as a baseline. **With these the drone flies reliably**. However, during demanding maneuvers such as flips or propwash manoeuvres, it becomes evident that there is room for improvement in terms of stability and responsiveness.

The tuning process was carried out on a larger 6-inch FPV drone provided by one of the team members. The drone was freshly flashed with Betaflight v12, meaning all filters and settings were at factory defaults. Achieving an optimal tune required several attempts due to challenges with filter adjustments and occasional logging issues. Nevertheless, the drone was successfully tuned, and the improvement was noticeable. During propwash manoeuvres, the drone remained significantly more stable, and flips were executed with greater precision at the stop.

Our tuning objective was to achieve a fast step response without overshoot, while maintaining good compliance and sensitivity. It is important to note that there is no universally “perfect” tune. Settings depend heavily on pilot preferences. Racing pilots for example maybe want more responsiveness for competitive performance, whereas freestyle pilots want smoothness and flight feel.

Since none of our team members are professional pilots, we focused on achieving robustness and a step response without overshoot. This tuning approach is probably somewhere between what a racing pilot and a freestyle pilot would prefer. Robustness in this context is important for practical scenarios, such as when a propeller is slightly bent, the battery sits slightly off center or when an additional component like a camera is mounted on the drone.

6.4 Decode

Text...

6.5 Code Structure and programming style

From the start of the project, we recognized that a clear and maintainable code structure would be essential. The initial proof-of-concept code already contained a rough separation into topics such as data I/O and step response, but everything was packed into a single file, making it difficult to navigate. Our first step was to comment out the proof-of-concept code and reorganize it into thematic sections. This was the foundation for a more organized layout and we came to an initial directory structure:

```
bf_controller_tuning/  
├── main.m  
├── data_io/  
│   ├── load_data.m  
│   ├── extract_header_information.m  
│   └── preprocess_data.m  
├── flight_parameters/  
│   ├── get_pid_parameters.m  
│   ├── scale_pid_parameters.m  
│   └── print_parameters.m  
├── spectrogram/  
│   ├── calculate_spectrogram.m  
│   ├── plot_spectrogram.m  
│   └── show_spectrogram.m  
├── spectra/  
│   ├── estimate_spectra.m  
│   ├── plot_spectra.m  
│   └── show_spectra.m  
├── frequency_response/  
│   ├── estimate_frequency_response.m  
│   ├── plot_bode.m  
│   └── show_frequency_response.m  
├── controller_analysis/  
│   ├── calculate_closed_loop.m  
│   ├── calculate_step_response.m  
│   └── show_controller_analysis.m  
└── utils/  
    ├── apply_rotfiltfilt.m  
    ├── downsample_frd.m  
    └── get_my_colors.m
```

This draft was never intended to be final. Rather, it served as an initial attempt to bring clarity and separation between different functional areas.

6.5.1 OOP vs. Functional Approach

At this stage, we discussed whether to implement the entire project using object-oriented programming (OOP) or stick to a modular, function-based approach. Initially, we considered OOP as a challenge, since most of us were not very experienced with it. We started writing some components as classes, but quickly realized that only one team member really was comfortable with OOP, while the others struggled. This made us reconsider. Our conclusion was to use OOP selectively—only for parts that benefit from reuse and encapsulation, such as data loading and plotting, while keeping the rest function-based for simplicity.

This hybrid approach allowed us to maintain flexibility without overcomplicating the design.

After several attempts, we agreed on the following principles:

- A main script (main.m) for user input, figure selection, filter settings, and PID tuning
- A core class to handle all calculations (essentially the logic from the proof of concept, but without plotting)
- A plot utilities class to centralize all plotting functions for consistency

Existing library functions from the proof of concept were reused without modification. The thematic grouping included:

```
bf_controller_tuning/  
├── class/  
│   ├── main_class.m  
│   └── plot_utils.m  
├── lib/  
│   ├── apply_rotfiltfilt.m  
│   ├── calculate_closed_loop.m  
│   ├── calculate_controllers.m  
│   ├── calculate_step_response_from_frd.m  
│   ├── calculate_transfer_functions.m  
│   ├── downsample_frd.m  
│   ├── estimate_frequency_response.m  
│   ├── estimate_spectra.m  
│   ├── estimate_spectrogram.m  
│   ├── expand_multiple_figure_nr.m  
│   ├── extract_header_information.m  
│   ├── get_chirp_signals.m  
│   ├── get_fcut_from_D_and_fcenter.m  
│   ├── get_fcut_from_exp.m  
│   ├── get_filter.m  
│   ├── get_ind_eval.m  
│   ├── get_my_colors.m  
│   ├── get_notch_Q.m  
│   ├── get_pid_scale.m  
│   └── get_switch_case_text_from_para.m  
├── logs/  
└── main.m
```

This structure worked well and kept the main script simple for the user. However, after a review meeting, our instructor suggested further modularization. The reason: we had created a “god class,” which limited flexibility. For example, if someone wanted to use the tool only for plotting spectrograms without any drone-specific logic, the current design made that difficult. Also the program was slow. All calculations were made, everytime the user changed something. We wanted to improve the speed in only doing these calculations again, which are relevant on that specific user input change.

6.5.2 Final Structure Concept

Out of this, we decided to split up this god class into three classes. The `flight_analyzer` class handles **frequency domain analysis and vibration characterization**. Its core responsibility is performing spectral analysis on flight log data to identify resonance frequencies and assess filter effectiveness. It computes power spectral density of gyro signals (both filtered and unfiltered) and generates spectrograms showing how vibration frequencies vary with throttle levels across all three axes (roll, pitch, yaw). This analysis is essential for identifying noise sources, evaluating filtering strategies, and optimizing drone stability during tuning. The `flight_data` class is responsible for **loading, parsing, and managing flight log data**. It provides methods to extract relevant information from Blackbox logs, such as gyro and motor data, and prepares this data for analysis. The `gyro_ctrl_tuning` class focuses on **tuning the PID controllers** for the drone's flight. It uses the results from the frequency response and spectral analysis to adjust the PID parameters, aiming to optimize the drone's flight characteristics. The `plot_utils` class remains responsible for **visualization and graphical representation** of all analysis results. Unlike the previous approach, this class is now more flexible and reusable. It can be used independently to generate plots from pre-calculated data, making it suitable for users who only want to visualize spectrograms, frequency responses, or step responses without running the full analysis pipeline. This modularity allows the plotting functionality to be easily adapted for different visualization needs and integrated into other workflows.

This approach still holds thematic sections, but is more accessible to use only specific parts of the tool.

```
bf_controller_tuning/
├── class/
│   ├── flight_analyzer.m
│   ├── flight_data.m
│   ├── gyro_ctrl_tuning.m
│   └── plot_utils.m
├── lib/
│   ├── apply_rotfiltfilt.m
│   ├── calculate_closed_loop.m
│   ├── calculate_controllers.m
│   ├── calculate_step_response_from_frd.m
│   ├── calculate_transfer_functions.m
│   ├── downsample_frd.m
│   ├── estimate_frequency_response.m
│   ├── estimate_spectra.m
│   ├── estimate_spectrogram.m
│   ├── expand_multiple_figure_nr.m
│   ├── extract_header_information.m
│   ├── get_chirp_signals.m
│   ├── get_fcut_from_D_and_fcenter.m
│   ├── get_fcut_from_exp.m
│   ├── get_filter.m
│   ├── get_ind_eval.m
│   ├── get_my_colors.m
│   ├── get_notch_Q.m
│   ├── get_pid_scale.m
│   └── get_switch_case_text_from_para.m
└── logs/
```

└─main.m

6.6 Conversion MATLAB to Python

Text...

6.7 Python in MATLAB

Text...

6.8 Transfer MATLAB Library to Python

Text...

7 Results

Text...

8 Discussion and next steps

8.1 Discussion

Text...

8.2 Next Steps

Text...

8.3 Conclusion

Text...

9 Directories